SUPERMARKET SALES & INVENTORY ANALYTICS AUTOMATION

Python • PostgreSQL • Power BI

A practical analytics project for automating CSV data preparation, database storage, and business reporting.

Project area	Data Analytics & Automation
Objective	Turn sales, product, and inventory files into analysis-ready data and interactive reports.
Tools	Python, pandas, SQLAlchemy, PostgreSQL, Power BI, GitHub
Report status	Documents the workflow and design. Final dashboard visuals and measured results remain to be validated.

Prepared as a portfolio project report

1. Executive Summary

The Supermarket Sales & Inventory Analytics Automation project is designed to help a small or medium-sized supermarket turn routine CSV exports into structured, analysis-ready data. Python validates and cleans files, PostgreSQL stores structured records, and Power BI presents sales and stock information through interactive reports.

The workflow aims to reduce repetitive manual preparation, improve basic data quality, retain an import history, and answer questions such as which products sell best, how sales change over time, and which products may need replenishment. The project includes Python modules for validation and cleaning, PostgreSQL tables for products, sales, and inventory, an import log, and analytical SQL views. The planned Power BI report contains two pages: Sales Performance and Inventory Management.

This report documents the project design and implementation components known to date. Final dashboard screenshots, verified KPI values, and measured automation benefits should be added after end-to-end testing.

2. Problem Statement

Retail sales and inventory data is often exported from point-of-sale (POS) systems into CSV or Excel files. Staff may need to clean inconsistent text, find missing values, remove duplicates, validate product references, calculate performance indicators, and prepare recurring reports manually. This can be time-consuming and may lead to inconsistent results.

The proposed solution places files in designated folders, checks and cleans them with Python, loads valid records into PostgreSQL, and uses Power BI to visualize sales performance and inventory health.

3. Objectives

• Automate ingestion of product, sales, and inventory CSV files from designated folders.

• Validate required columns and reject unreadable, empty, or structurally invalid files.• Trim text, parse dates, convert numeric columns, and remove exact duplicate rows.

• Store structured data in PostgreSQL for querying and analysis.

• Maintain an import log and use file hashes to help identify repeated imports.

• Create SQL views for sales/product analysis and inventory status.

• Design two Power BI pages for sales performance and inventory management.

• Publish reproducible project assets to GitHub without exposing credentials or private data.

4. Tools and Technologies

Technology	Role
Python	Orchestrates file processing and validation/cleaning rules.
pandas	Reads CSV files, transforms tabular data, converts data types, and removes duplicates.
SQLAlchemy	Manages the Python-to-PostgreSQL connection and database operations.
PostgreSQL	Stores product, sales, inventory, and import-log records; supports SQL views.
Power BI Desktop	Builds interactive business reports and visualizations.
Jupyter / Anaconda	Provides the Python development and testing environment.
Git / GitHub	Version control, documentation, and portfolio publication.

5.System Architecture

The project workflow consists of five stages:

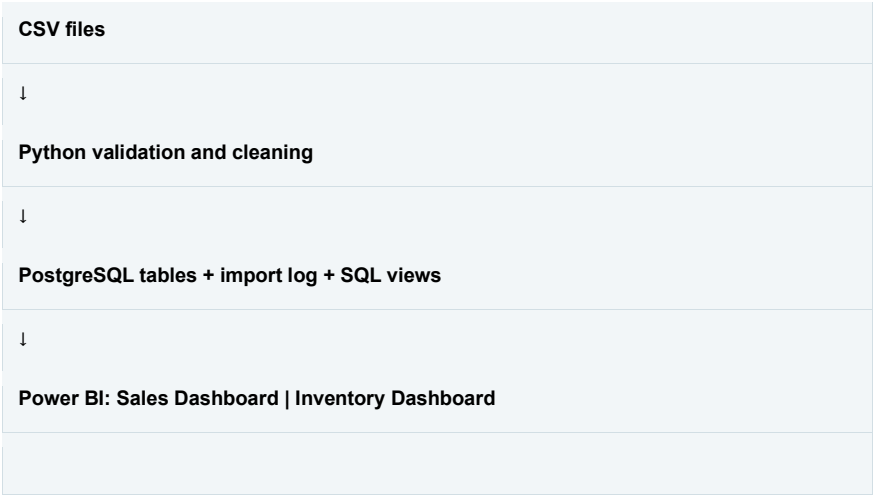
• **Source files:** Product, sales, and inventory CSV exports are placed in their corresponding incoming folders.

• **Validation:** Python checks file readability, required columns, empty files, and missing values in required fields.

• **Cleaning:** pandas trims text, parses dates, converts numeric columns, removes exact duplicate rows, and applies checks before loading.

• **Database:** PostgreSQL stores records and import history. SQL views combine transaction and product data and classify stock status.

• **Reporting:** Power BI presents sales trends, product/category comparisons, payment methods, stock availability, and reorder alerts.



6.Data Model

The core data model contains three business tables and one operational import log.

Table/view	Purpose	Representative fields
products	Product master data	product_id, product_name, category, cost_price, selling_price, reorder_level
sales	Sales transaction lines	bill_number, sale_date, product_id, quantity, discount, payment_method
inventory	Inventory records/snapshots	inventory_date, product_id, opening_stock, stock_received, damaged_units, stock_count, closing_stock
file_import_log	Tracks imported files	file_name, file_hash, import_status, rows_imported
vw_sales_analysis	Sales joined to product details	product_name, category, quantity, sales/cost/profit fields; discount logic must be verified
vw_inventory_analysis	Inventory joined to product details	closing_stock, reorder_level, stock_status, suggested_reorder_quantity

Relationship design: products is the product dimension. sales and inventory both reference product_id. In Power BI, the usual model is products (1) to sales (*) and products (1) to inventory (*). A 1:1 relationship is appropriate only if product_id is unique in both tables; a direct sales-to-inventory relationship is generally unnecessary.

5. Python Data Processing

5.1 Validation

The validation module checks required columns for each file type. It rejects unsupported file types, unreadable CSV files, empty files, missing columns, and missing values in required fields.

5.2 Cleaning

The cleaning module trims whitespace from text values, parses sale_date and inventory_date, converts selected fields to numeric types, and removes exact duplicate rows. It reports the row count before cleaning, duplicates removed, and row count after cleaning.

5.3 Pipeline orchestration

The main pipeline is intended to scan incoming folders, validate and clean each CSV, check business rules and product references, append valid rows to PostgreSQL, log successful imports, and move files into processed or rejected folders. SHA-256 file hashes can help recognize previously imported files.

Testing note: Run an end-to-end test with controlled sample files before claiming the full pipeline is production-ready. Confirm handling of duplicate product IDs, failed database inserts, and retries.

6. SQL Analysis and Business Rules

The sales analysis view joins sales records to product details to support sales and product analysis. The inventory analysis view joins inventory records to product details and assigns stock categories such as Out of Stock, Low Stock, and Available.

Before publishing revenue and profit KPIs, confirm the meaning and unit of the discount field. A percentage discount and a fixed-currency discount require different calculations. Verify the calculation against a known transaction.

Business questions supported by the design include:

• Which products and categories generate the highest sales?

• How do sales change by day, week, or month?

• Which payment methods are used most often?

• Which products are below their reorder levels?

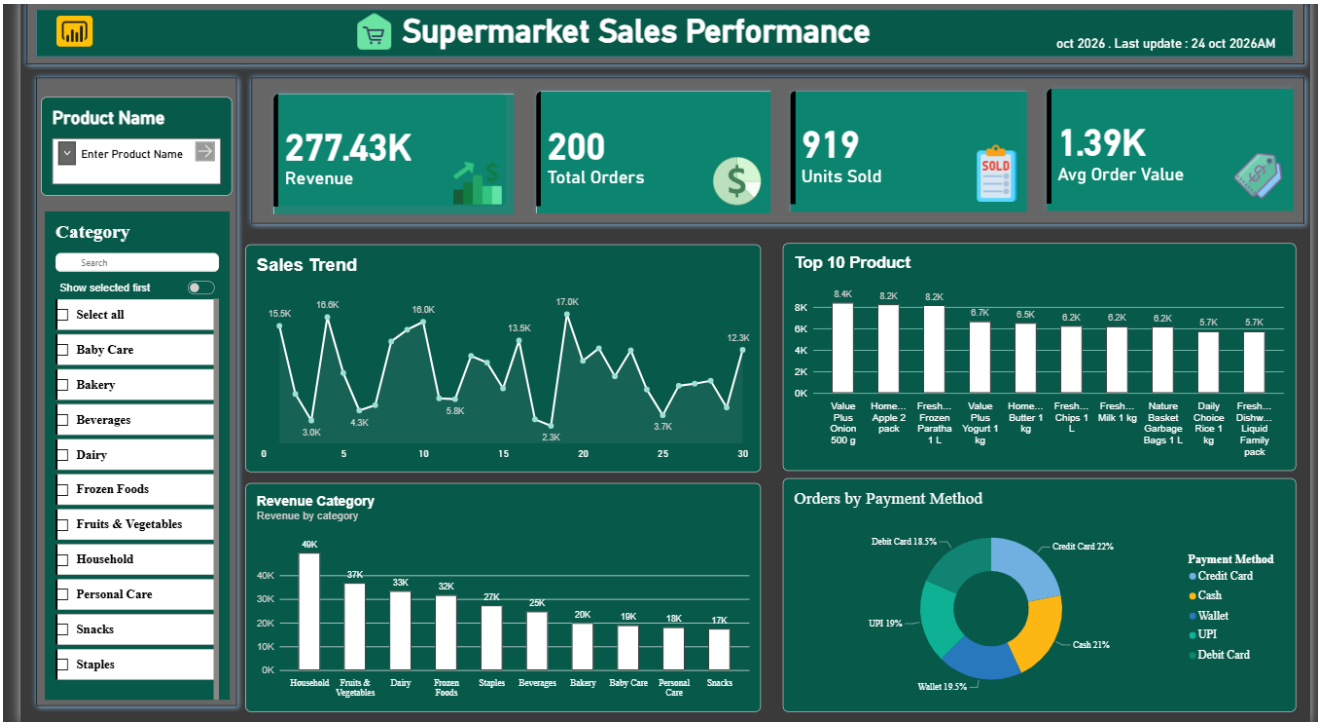
• How much stock is available, received, or damaged?

• Which products appear most profitable after validating cost and discount rules?

7. Power BI Dashboard Design

Page 1 — Sales Dashboard

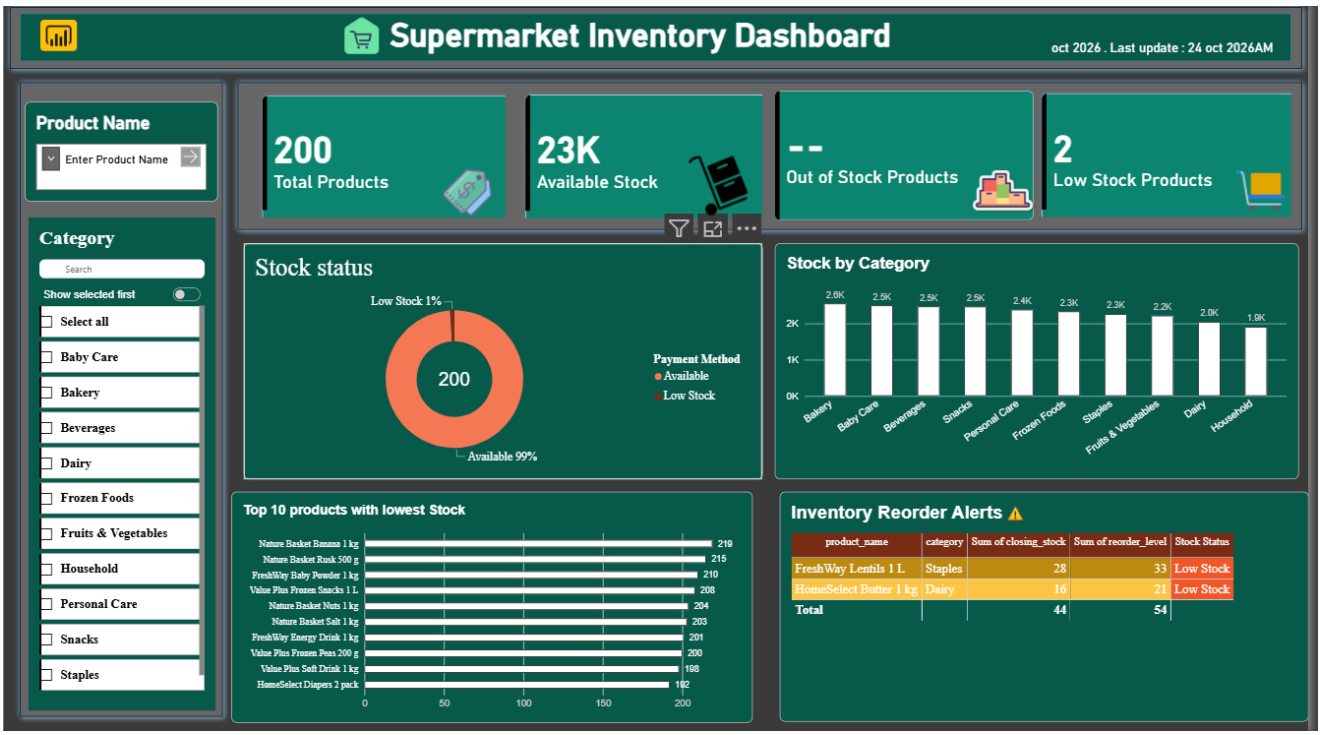
Recommended KPI cards: Total Revenue, Total Orders, Units Sold, and Average Order Value. Recommended visuals: sales trend over time (line chart), Top 10 products by revenue (bar chart), revenue by category (column chart), payment method distribution (donut chart), and a product detail table. Useful slicers include date, category, and payment method.



Page 2 — Inventory Dashboard

Recommended KPI cards: Total Products, Available Stock, Low Stock Products, and Out-of-Stock Products. Recommended visuals: stock status breakdown (donut chart), closing stock by product (bar chart), stock by category (column chart), and a reorder-alert table containing product name, closing stock, reorder level, and stock status. If reorder_level is stored in products, use the related field or a suitable measure.

Design guidance: use a 16:9 canvas, clear titles, aligned KPI cards, readable labels, consistent number formatting, and restrained colors. A local Power BI report connected to CSV files refreshes from those file paths; it does not automatically refresh from PostgreSQL unless configured separately.



8. Testing and Validation Plan

Test	Expected result	Status
Valid CSV	Passes validation and loads expected rows	To verify
Missing required column	Rejected with a clear message	To verify
Empty/unreadable CSV	Rejected without database insert	To verify
Repeated file hash	Skipped or safely handled without duplicate import	To verify
Unknown product_id	Rejected or flagged before loading	To verify
Power BI refresh	Reloads data from the selected source files	To verify

9. Limitations and Assumptions

- Incoming files are assumed to follow the expected CSV structure and column names.
- The discount calculation must be confirmed before revenue and profit results are treated as final.
- Final dashboard screenshots and KPI values are not included because the dashboard is still being completed.
- Power BI using CSV files will not automatically reflect PostgreSQL changes unless the source or refresh workflow isconfigured accordingly.
- The pipeline should be tested with duplicate product IDs and repeated imports to verify primary-key and retry handling.

10. Recommended GitHub Structure

```
Supermarket_Automation/
python/
main.py
validate_data.py
clean_data.py
sql/
create_tables.sql
analysis_views.sql
sample_data/
products.csv
sales.csv
inventory.csv
powerbi/
supermarket_dashboard.pbix
reports/
Supermarket_Sales_Inventory_Automation_Project_Report.pdf
README.md
requirements.txt
.gitignore
```

Keep passwords, local connection strings, private exports, and sensitive data out of GitHub. Include sanitized sample CSVs and use environment variables for credentials.

11. Conclusion

This project combines Python-based data validation and cleaning, PostgreSQL storage and SQL views, and a two-page Power BI reporting design. It provides a practical foundation for reducing repetitive data preparation and improving visibility into sales and inventory. Before presenting it as complete, validate the pipeline end to end, confirm discount and profit calculations, finish both Power BI pages, capture screenshots, and update the repository with tested results.

Appendix A — Suggested requirements.txt

```
pandas
SQLAlchemy
psycopg2-binary
```

Install dependencies in the same Python environment used to run the automation, and include any other packages imported by the final code.